

D3XX Library v1.4.0.6

Compatibility

D3XX library is compatible with the following Windows operating systems:

- Windows 11 x64, ARM64
- Windows 10 x64, x86

Package Contents

The files included in this package are:

- D3XX interface header
 - FTD3XX.h
- Dynamic library files
 - x86: FTD3XXWU.dll, FTD3XXWU.lib, FTD3XXWU.exp, FTD3XXWU.pdb
 - x64: FTD3XXWU.dll, FTD3XXWU.lib, FTD3XXWU.exp, FTD3XXWU.pdb
 - ARM64: FTD3XXWU.dll, FTD3XXWU.lib, FTD3XXWU.exp, FTD3XXWU.pdb
- Static library files
 - x86: FTD3XXWU.lib, FTD3XXWU.pdb
 - x64: FTD3XXWU.lib, FTD3XXWU.pdb
 - ARM64: FTD3XXWU.lib, FTD3XXWU.pdb

Releases

1.4.0.6 (August 5, 2026)

- Fixed notification callback reliability when notification endpoint traffic runs concurrently with data traffic on other channels.
- Prevented callback-thread self-deadlock during callback clear and close operations.
- An application can now link D2XX and D3XX together. The seven function names common to both libraries are exported additionally as **FT3_*** aliases, and **FTD3XX.h** redirects them automatically, so the two import libraries no longer produce duplicate symbols. Existing **FT_*** export ordinals are unchanged, so applications using only D3XX are unaffected. See “Using D2XX and D3XX in the Same Application” below.

1.4.0.1

- Addressed the behavioral differences between the D3XX library and the WinUSB-based D3XX libraries.

Using D2XX and D3XX in the Same Application

A single application can use both libraries — for example to drive an FT600/FT601 over D3XX while talking to an FT232/FT2232 over D2XX.

Seven function names exist in both libraries and previously caused duplicate symbol errors when both import libraries were linked:

- `FT_Close`
- `FT_CreateDeviceInfoList`
- `FT_GetDeviceInfoList`
- `FT_GetDeviceInfoDetail`
- `FT_GetDriverVersion`
- `FT_GetLibraryVersion`
- `FT_ListDevices`

The D3XX library exports each of these additionally as an `FT3_*` alias, and `FTD3XX.h` declares the alias and redirects the original name to it:

```
FT_STATUS WINAPI FT3_Close(FT_HANDLE ftHandle);
#define FT_Close FT3_Close
```

No source change is required. Code including `FTD3XX.h` keeps calling `FT_Close`, and the call resolves to `FT3_Close` in `FTD3XXWU.dll`. Code including `ftd2xx.h` resolves to `FT_Close` in `FTD2XX.dll`. All other D3XX function names are unique to D3XX and need no alias.

Requirement: keep the headers in separate source files

`FTD3XX.h` and `ftd2xx.h` must not be included in the same translation unit. Both headers independently declare `FT_STATUS` values, `FT_DEVICE`, `FT_DEVICE_LIST_INFO_NODE` and the `FT_FLAGS_*` constants, so including both produces type redefinition errors (C2365, C2371). The function-name aliasing does not affect these types.

Place the code for each library in its own source file and expose a neutral interface between them:

```
d2xx_module.cpp    #include "ftd2xx.h"      -> links FTD2XX.lib
d3xx_module.cpp    #include "FTD3XX.h"    -> links FTD3XXWU.lib
main.cpp           includes neither header, calls both modules
```

Link both import libraries as usual. Verified for Win32; the same applies to x64 and ARM64.

Diagnostic Tracing

The library is built with WPP software tracing enabled. Traces can be captured on an end-user machine without a special build, which is useful when reporting an issue to FTDI.

- Trace provider GUID: {5953494A-3018-4DC5-8148-FD57D9AED2D3}

Capture a trace:

```
logman create trace ftd3xxwu -p "{5953494A-3018-4DC5-8148-FD57D9AED2D3}" 0xffffffff 0xff -o ftd3xxwu.etl
logman start ftd3xxwu
... reproduce the issue ...
logman stop ftd3xxwu
```

Decoding requires the trace message format data, which is contained in the `FTD3XXWU.pdb` files shipped with this package. Using the Windows Driver Kit tools:

```
tracepdb -f FTD3XXWU.pdb -p .\tmf
tracefmt -display -p .\tmf ftd3xxwu.etl
```

If you cannot decode the trace locally, send the `.etl` file to FTDI support along with the library version.

Limitations

- When streaming is enabled and asynchronous `FT_ReadPipe` is used, the size parameter passed to read the data should be aligned to 1024 bytes. This is a requirement enforced by WinUSB.
- `FT_GetDriverVersion` returns the same version as Library Version because this package does not use any FTDI drivers.